

JWT : JSON Web Token

1. Definition and Purpose of JWT

JWT (JSON Web Token) is a secure token format used for:

- **Authentication** — verifying the identity of the user
- **Authorization** — validating whether the user has permission to access a resource

JWT tokens are **digitally signed**, meaning they **cannot be altered without detection**. Therefore, they are trusted across distributed systems such as **microservices architectures**, cloud applications, and APIs.

JWT is commonly used because it is:

- Stateless
- Lightweight
- Self-contained

The server does not need to store session data; the token itself contains the claims required for authorization.

2. Structure of JWT

A JWT token contains **three parts**, separated by dots:

Header.Payload.Signature

2.1 Header

Defines token metadata, including:

- Token type → typ = JWT
- Signing algorithm → commonly HS256

Example (JSON format):

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

This is Base64 encoded.

2.2 Payload

The payload contains **claims**, i.e., information about the user and token.

Typical claims include:

- User identity
- Email
- Role
- Expiration time
- Issuer information

This part is also Base64 encoded.

2.3 Signature

The signature ensures **token integrity**.

It is generated using:

Header + Payload + Secret Key

If even **one character changes inside the token**, the signature becomes invalid.

Therefore:

- Token modification = token rejection
-

3. JWT Authentication Flow (Step-by-Step)

JWT authentication follows a standardized workflow.

Step 1 — Client Sends Login Request

The client submits credentials:

username + password

to the **Authentication Server**.

Step 2 — Authentication Server Validates Credentials

If the credentials are valid:

- A JWT token is generated
- The token is signed using a **secret key**

The token is returned to the client.

Step 3 — Client Stores Token

Token storage options:

- Local storage (web)
 - Session storage
 - Secure storage in mobile apps
-

Step 4 — Client Sends Token with Each Request

Token is sent in the HTTP header:

Authorization: Bearer <token>

Step 5 — Resource Server Validates Token

The server performs:

- Signature validation
- Expiry check
- Issuer verification
- Audience verification

If valid → **Access granted**

If invalid → **401 Unauthorized**

This makes JWT ideal for **stateless distributed systems**.

4. JWT Claims

Claims are the data embedded inside the token.

4.1 Registered Claims (Standard)

Claim	Meaning
iss	Issuer
aud	Audience
exp	Expiry time
sub	Subject
iat	Issued At
jti	Token ID

These improve security and standardization.

4.2 Custom Claims

Examples include:

- Name
- Email
- Role
- UserId

These support application-specific authorization logic.

5. Security Best Practices for JWT

To maintain secure authentication:

- Always use **HTTPS**
- Set a **token expiry time**
- Never store passwords inside the token
- Use a **long and secure secret key**
- Validate **issuer and audience values**
- Never expose secret keys publicly

JWT should **not store sensitive data** such as passwords or confidential identifiers.

6. JWT in Microservices Architecture

In microservices:

- **Authentication is centralized** via an Identity Service
- The Identity Service issues a token
- Each service validates the token independently
- Users authenticate once

Centralized Flow

Client → Auth Service → JWT Issued

Client → API Gateway → Services validate JWT

The gateway or each service verifies:

- Signature
- Expiry
- Claims

This prevents repeated logins and ensures distributed trust.

7. Token Validation Process (Critical)

When a token is received:

1. Read token from Authorization header
2. Decode Header & Payload
3. Recompute Signature using secret key
4. Compare computed signature with token signature
5. If matched → token valid
6. If not → reject request

Unauthorized requests return **401 Unauthorized**.

8. JWT Implementation Workflow in .NET Core

8.1 Token Configuration

Store configuration such as:

- Key
- Issuer
- Audience

inside appsettings.json.

8.2 Configure Authentication Middleware

Using:

```
AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
```

This enables JWT validation.

8.3 Token Creation

Token is generated using:

- Claims
- SymmetricSecurityKey
- SigningCredentials

The token contains identity, roles, and metadata.

8.4 Securing Controllers

Add [Authorize] attribute.

If unauthorized, the API returns:

401 Unauthorized

8.5 Testing via Postman

Process:

1. Call login endpoint — receive token
 2. Send token in Authorization header
 3. Access secured API successfully
-

9. Why JWT is Ideal for Microservices

JWT works effectively in microservices because:

- Tokens are stateless
- Services validate tokens independently
- Centralized authentication does not create bottlenecks
- Cross-service trust is maintained

This model eliminates the need for **shared session storage**.
